Hong Kong Metropolitan University
MSc of Computing

COMP8090SEF — Data Structures and Algorithms
Course Project — Task 1

Library Borrowing System

WONG Yan-ho, Michael

Student ID: 14194311

Outline

1. Introduction and System Overview
2. System Architecture
3. OOP Concepts and Implementation
4. Core Functions and Feature
5. Live Demonstration and Test Cases
6. Self-Reflection and Future Improvement

Introduction and System Overview

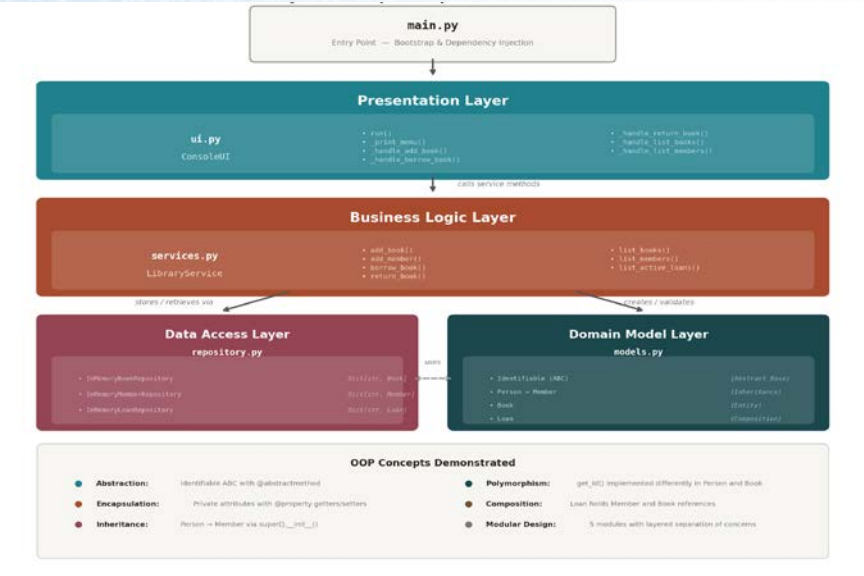
Problem Statement:

- Develop a console-based Library Borrowing System using Python
- Demonstrate software engineering principles through OOP
- Handle book inventory, member registration, and loan tracking

Solution Approach:

- Layered architecture with separation of concerns
- Five Python modules: main, models, repository, services, UI
- All five OOP concepts: Abstraction, Encapsulation, Inheritance, Polymorphism, Composition
- Built using only the Python Standard Library

System Architecture



Four-Layer Design

Presentation (ui.py)

Console menu navigation, input validation, formatted output

Business Logic (services.py)

Borrowing rule, eligibility check, loan management

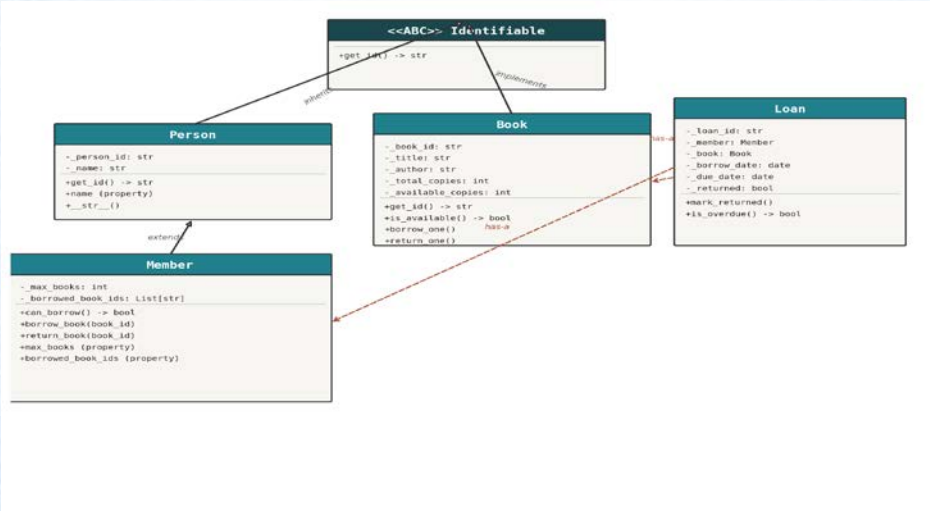
Data Access (repository.py)

In-memory storage with dictionary-based lookup

Domain Model (models.py)

Entity class: Person, Member, Book, Loan, Identifiable

OOP Concept and Class Hierarchy



Abstraction

Identifiable ABC
abstract get_id()

Encapsulation

Private attrs with
@property getters

Inheritance

Person → Member
class hierarchy

Polymorphism

get_id() on Person
and Book

Composition

Loan holds Member
+ Book refs

Book and Member Management

Book Management

- Register books with unique ID, title, author, copy count
- Real-time availability tracking (available vs total copies)
- Auto-adjust copy count during borrow or return
- Encapsulation via @property with validation logic

Member Management

- Member registration with unique ID and names
- Configurable borrowing limit
- Active loan tracking per member
- Eligibility validation before approving loan
- Member extends Person

Loan Operation and User Interface

Loan Operation

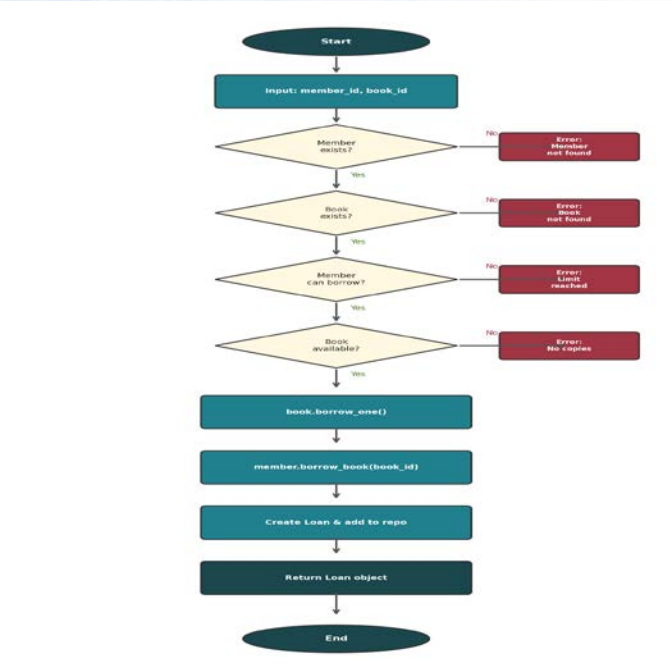
- 4-step validation: member, book, eligibility, availability
- Configurable loan period
- Due date tracking and overdue detection
- Return processing with automatic status update
- Complete loan history maintenance

Console User Interface

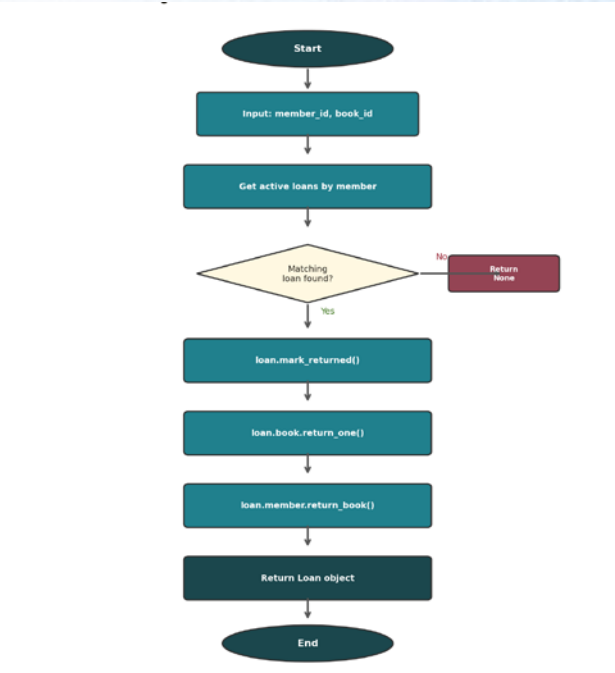
- Menu-driven navigation with 7 operations + Quit
- Input validation and error handling
- Formatted output for books, members, loans
- Exception handling with user-friendly messages

Process Flow: Borrow and Return

Borrow Book Flow



Return Book Flow



Data Structure Used

Data Structure	Python Type	Location	Purpose
Dictionary	Dict[str, Book]	BookRepository._books	O(1) lookup by book_id
Dictionary	Dict[str, Member]	MemberRepository._members	O(1) lookup by member_id
Dictionary	Dict[str, Loan]	LoanRepository._loans	O(1) lookup by loan_id
List	List[str]	Member._borrowed_book_ids	Track active borrows
Integer	int	LibraryService._next_loan_id	Auto-increment loan IDs
Date	datetime.date	Loan._borrow_date, _due_date	Temporal tracking / overdue

Demonstration: Normal Operations

Test	Operation	Result
TC-01	Add Book	4 books registered with unique IDs and copy counts
TC-02	Add Member	3 members registered with borrowing limits
TC-03	Borrow Book	3 loans created with 14-day due dates
TC-04	List Active Loan	All 3 active loans displayed correctly
TC-05	Book Availability	Copy counts adjusted: B001 shows 1/3 available
TC-06	Return Book	Loan L0001 marked returned, B001 restored to 2/3

Demonstration: Error Handling

Test	Scenario	Expected Result
TC-07	Duplicate Book ID	Error: Book ID already exists
TC-08	Duplicate Member ID	Error: Member ID already exists
TC-09	Borrowing Limit Reached	Error: Maximum borrowing limit
TC-10	No Available Copies	Error: No available copies
TC-11	Non-existent Member	Error: Member not found
TC-12	Non-existent Book	Error: Book not found
TC-13	Return Non-existing Loan	Return result: None
TC-14	Member Status After Ops	Correct borrowed counts per member
TC-15	Overdue Detection	Backdated loan: is_overdue() → True
TC-16	Final State Verification	All books, members, loans consistent

Self-Reflection and Future Improvement

Current Weakness

- In-memory storage only — data lost on exit
- No user authentication or role-based access
- Limited search (exact ID lookup only)
- Console-only interface
- No concurrent access handling

Proposed Improvement

- Integrate SQLite or JSON file persistence
- Add login system with librarian/patron roles
- Implement keyword search by title/author
- Build GUI using Tkinter
- Client-server architecture for concurrency

Thank You